

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Software vulnerabilities

Michele Colajanni

Dipartimento di Informatica – Scienza e Ingegneria

michele.colajanni@unibo.it

Fundamental principles (*State of the situation*)

1) **Threat and risk analysis models**: "who the attacker is, what they want, and what the attack surfaces are" remains the same, regardless of the technology.
2) **Fundamental vulnerability classes**: ***user-related vulnerabilities***, ***software-related vulnerabilities*** (buffer overflows, code injection, race conditions, memory management issues) remain the same. The languages in which they manifest may change, but the underlying logic does not. The same for ***system*** and ***management vulnerabilities***.
3) **Cryptography, its mathematical primitives (RSA, elliptic curves, hashes), and associated protocols**: implementations change, but the underlying theory does not. [With the exception of *quantum-safe cryptography*.]
4) **Secure design principles**: **least privilege**, **strong authentication**, **defense-in-depth**, and **fail-safe defaults** (explicit permissions rather than default ones) remain perfectly valid in any architecture.



The unsaid truth

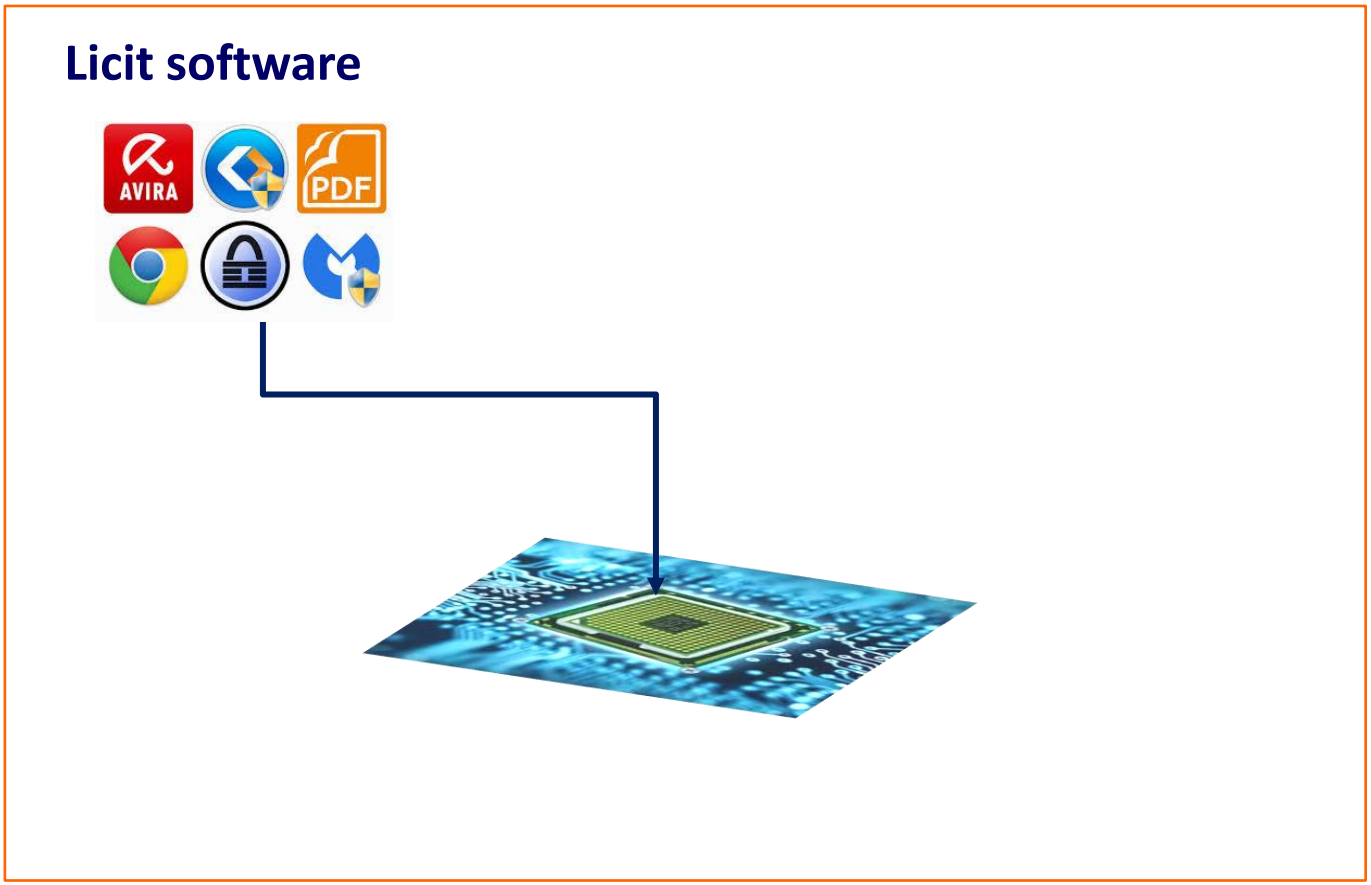
If you have a machine that by design can behave differently after production, then you could have expected high sensitivity on security and safety. **Not at all !**

1. At least 90% of cyber vulnerabilities are due to **software errors**
2. In the industrial world, there are no other examples of producers that have been able to convince customers that ***buying*** prototypes and ***patching*** them every week/month is acceptable
3. Opening an attached file containing malware is not a fault of a “stupid” user. The problem is caused by *vulnerable software*



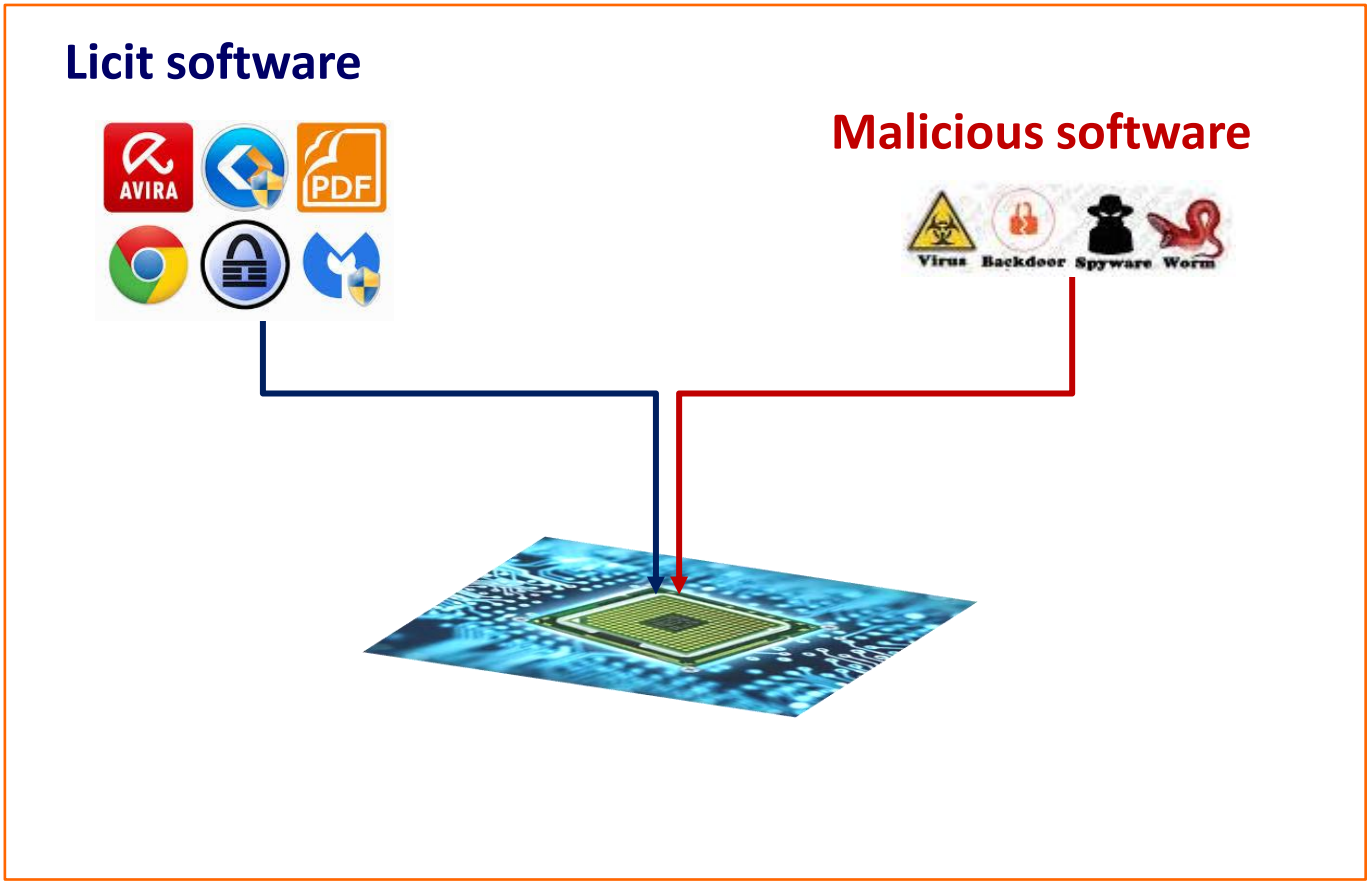
Software threat vector (1/3)

Computers are the only machines that by design can behave differently after production



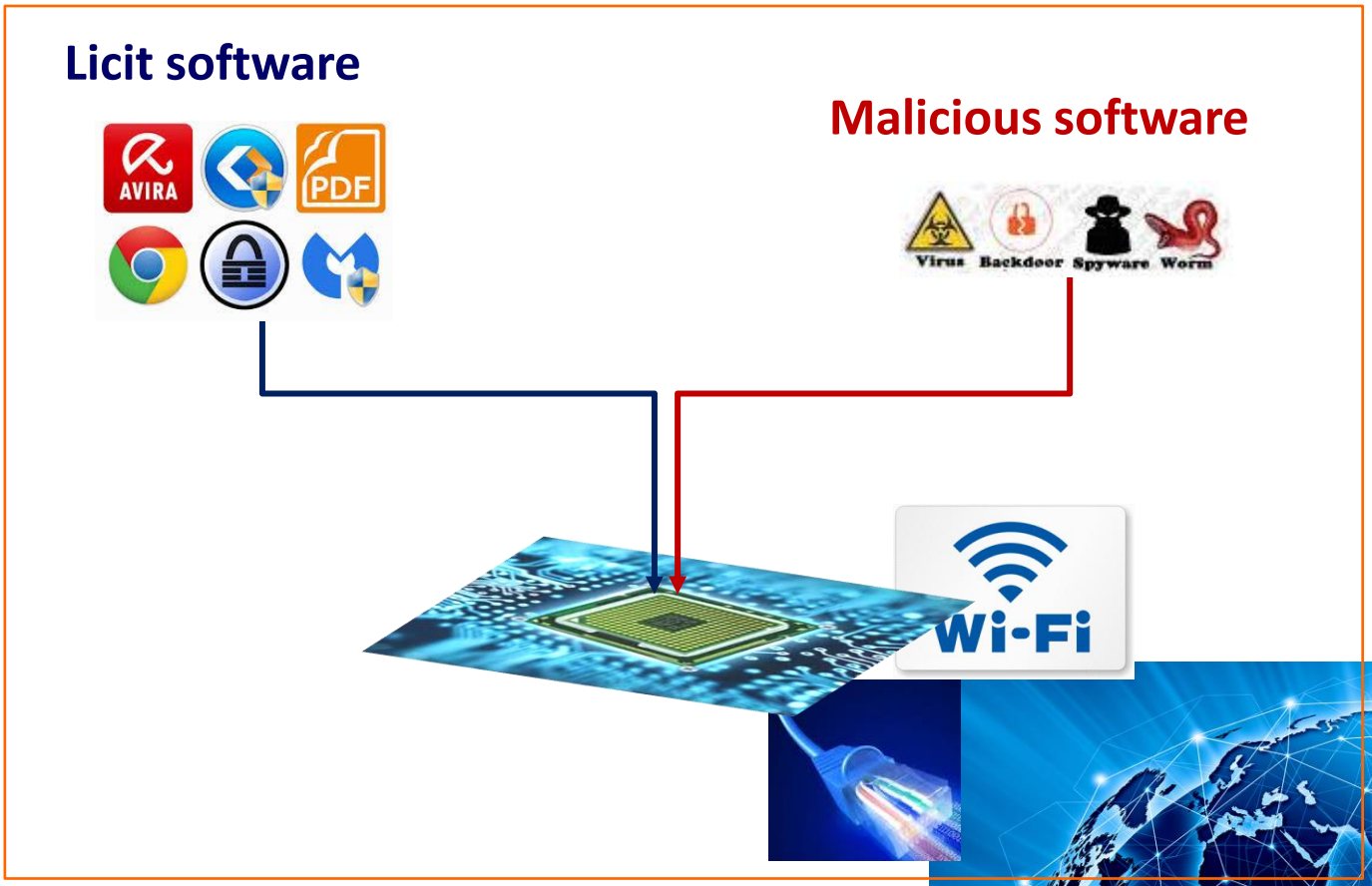
Software threat vector (2/3)

Computers are the only machines that by design can behave differently after production



Software threat vector (3/3)

- Computers are the only machines that by design can behave differently after production, and we connected all of them to the world!



CLICK HERE TO KILL EVERYBODY



Software is one of the most serious vulnerabilities

"Despite their myriad manifestations and different targets, **nearly all cyber attacks** have one fundamental cause: **the code used to run too many systems and services is insecure**.

Flaws in its design, implementation, testing, and operations allow attackers all-too-easy access".



Some statistics

- **99%** of the security vulnerabilities exploited in 2020 will continue to be those known to the organization at the time of exploit [Gartner]
- **79%** of the companies that experienced a security breach indicated they could have been avoided it through a *patch process* or a *configuration change* [Voke]
- The average time to exploit a known vulnerability in recent years has decreased from **45 down to 3 days** [IBM]
- …



The two sides of software vulnerabilities

Software providers → *Vulnerable software*

- insecure coding
- design flaw
- implementation issues (reuse of unchecked components)
- insufficient testing and lack of audit trail
- lack of (immediate) software patches after vulnerability disclosure

Software users → *Management issues*

- use of unknown software components (download, install and hopefully patch)
- use of illegal software (not licensed and not maintained)
- incompetent installation
- lack of a vulnerability and patch management process (*patching software in enterprise systems ≠≠ patching a PC*)



Vulnerable software

If you have a “machine” that by design can have different behaviors after production, then you could have expected top sensitivity on security and safety. **Not at all !**

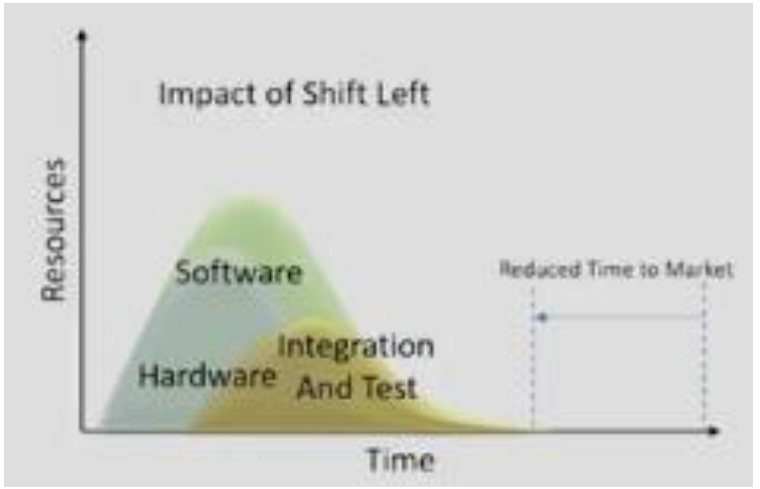
1. About 90% of cyber vulnerabilities are due to **software errors**

2. In the industrial world, there are no other examples of producers
   - that do not take any responsibility about the use of their products
   - that have been able to convince customers that ***buying*** software prototypes and ***patching*** them every week/month is normal

3. The truth is that opening an attached file containing malware is not a fault of a “stupid” user. **The problem exist only because the *software is vulnerable*, and a multi-billionaire industry exists because of this situation**

1. - Device and software producers are “shifting left”

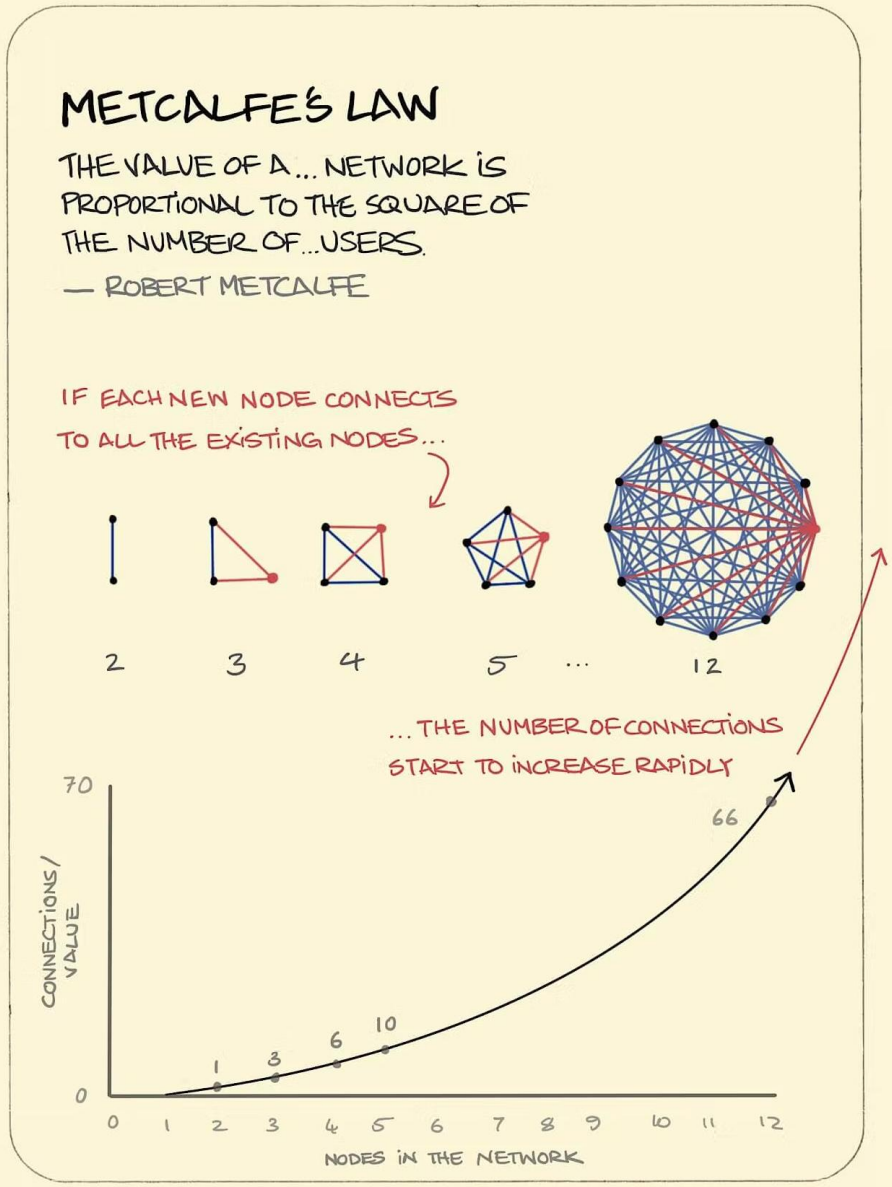




Reduced Time-To-Market ➔
More intrinsic vulnerabilities

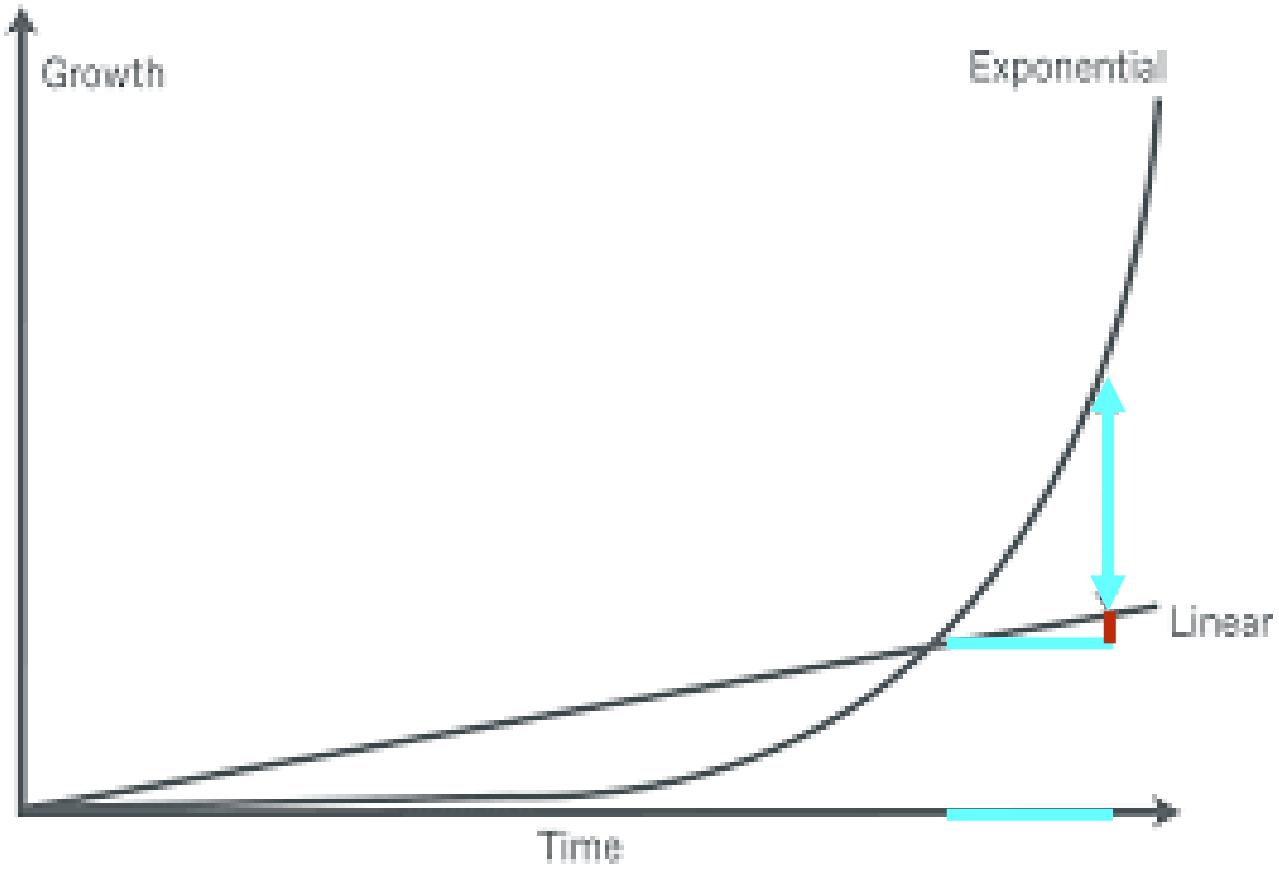
1) TIME TO MARKET

2. - The first “law” of the cyber space



2) MARKET COMPETITION

Exponential times prevent you from reaching the first
(A new version of the “paradox”)







Consequence

It is fundamental to offer the first "service", not the best!



If not all, this is true for the largest part of the market

3. - *Aleph One*, Bugtraq posting (December 1998)

1. Programmers do not think “multi−user”
2. Programmers are human, and humans are *lazy*. Thus, programmers will often use the “easy” approach instead of the most secure approach − and once it works, they often fail to fix it later
3. Most programmers are simply not good programmers
4. Most programmers are not security people; they simply don’t think like an attacker does: **linear *vs* network, vertical *vs* global thinking**
5. Most computer security models are terrible
6. There is lots of “broken” legacy software still integrated in modern systems. Fixing this software (to remove security faults or to make it work with more restrictive security policies) is difficult
7. Consumers don’t care about security
8. Security requires extra development time
9. Security costs in terms of additional testing (and there is no time for using formal verification methods)

3) COMPETENCES

Our fault

- There are few academic curricula that address software cybersecurity
- Even when there is a cybersecurity curriculum, they often do not discuss how to write *secure programs* but focus on A&A, cryptography, protocols, architectures and risk management
- Hence, even programmers who go through colleges and universities are unlikely to learn how to write secure programs by facing real issues such as *buffer overflows*, *string formatting*, *input checking*, …
  This is one of the most critical issues because the security of the cyberspace depends on those people



Software vulnerabilities

Poor code quality leads to unpredictable behavior.

- From a user's perspective, it often manifests itself as poor performance, usability and some errors
- From an attacker's perspective, it provides an opportunity to stress and penetrate the system in unexpected ways



Software and bugs

All software has bugs. Some of them are functional, while some:

- create **elevated privileges**
- cause **information leakage**, and/or
- favor **denial of service** attacks

These bugs are what we refer to as **software security vulnerabilities**

- These vulnerabilities are written into the software as a **coding errors**. While the vendors hopefully do not intend to release software with vulnerabilities, after the software is released for widespread use, they are *eventually* found



Software vulnerabilities can create a lot of issues

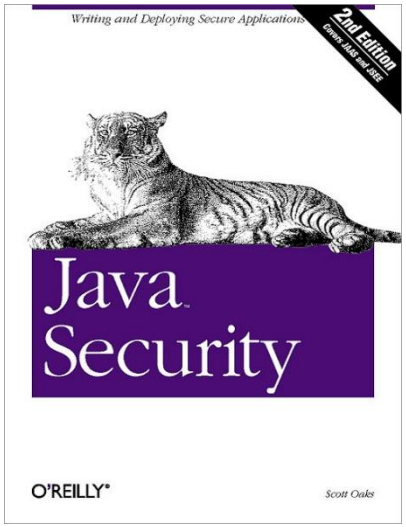
- When a process reads input from an attacker, the process may be exploited if it contains vulnerabilities

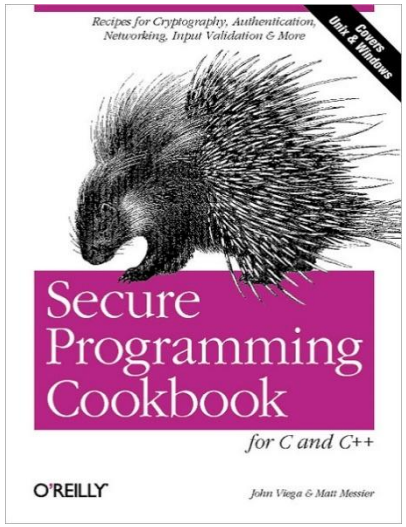
- When an attacker successfully exploits a vulnerability, he can
  1) Crash programs: *Compromises availability*
  2) Execute arbitrary code: *Compromises integrity*
  3) Obtain sensitive information: *Compromises confidentiality*

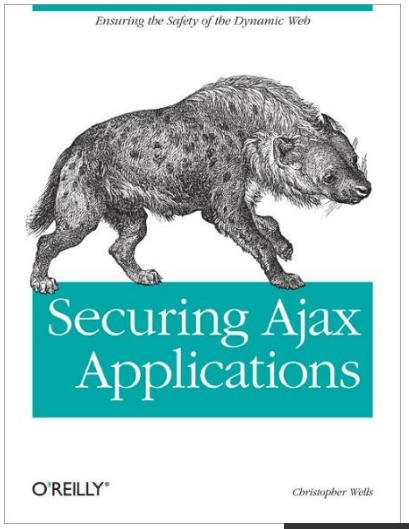
- Software vulnerability enables the attacker to run with privileges of other users, often *violating access control policies*

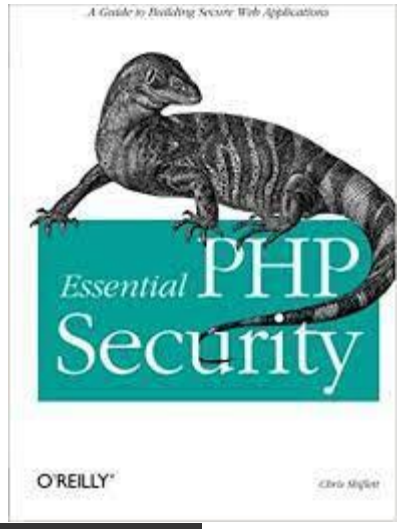


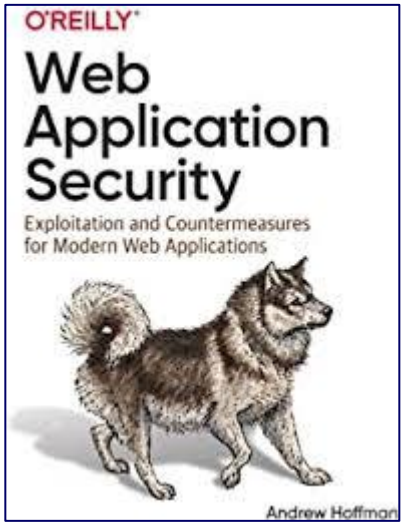
Secure programming –
Excuses for students, not for professionals

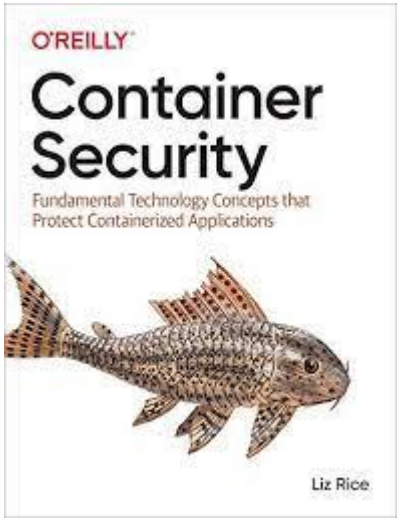


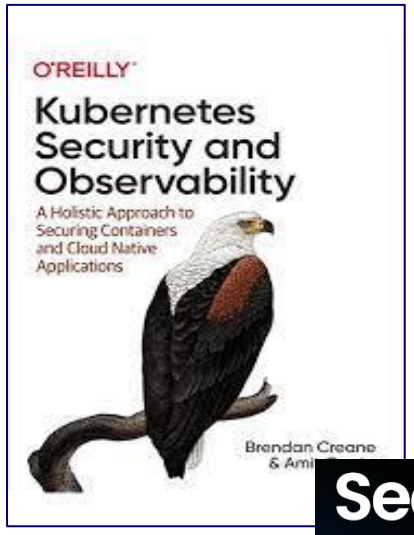


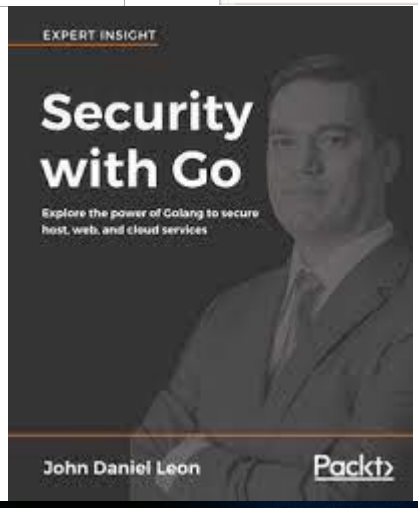












Secure coding training for the AI-assisted coding era

Sources of Information



The overall picture



Main difference

- **CWE** stands for **Common *Weakness* Enumeration**, and has to do with the vulnerability — **not the instance within a product or system**
- **CVE** stands for **Common *Vulnerabilities* and Exposures**, and has to do with the specific instance within a product or system — **not the underlying weakness flaw**

Weaknesses are errors that can lead to vulnerabilities

- A *software vulnerability* is a mistake in software that can be directly used to gain access to a system or network
- A *hardware vulnerability* is a mistake in hardware or its firmware that can be used to gain remote or physical access to a system



Common Weakness Enumeration (CWE)

- **CWE™** is a community-developed list of software and hardware weakness types
- It serves as a common language, a measuring stick for security tools, and as a baseline for
  - weakness identification
  - mitigation efforts
  - prevention efforts



CWE was launched in 2006 with the aim at creating specific definitions for each common weakness type. By leveraging the widest possible group of interests and talents, the hope is to ensure that each item in the list is adequately described and differentiated



https://cwe.mitre.org/

- **The objective is to identify, integrate and effectively describe common software weaknesses known to the industry and software assurance community**
- It leverages a taxonometric approach for **list integration**:
  - Identify and review dozens of existing taxonomies
    - Academic and professional: Aslam, RISOS, Landwehr, Bishop, Protection Analysis, etc.
    - High level lists: OWASP Top 10, 19 Deadly Sins, WASC, etc.
    - In-depth practical: PLOVER, CLASP, 7 Pernicious Kingdoms
  - Integrating taxonomies
    - Normalizing and deconfliction
    - Finding a proper balance between breadth and depth

Common Weakness Enumeration (CWE)

CWE™ is a living effort with ongoing work to capture the specific effects, behaviors, exploit mechanisms, and implementation details within the CWE list as well as to review and revise the presentation approaches to provide those that best suit the community using this information

The overall goal is to improve the quality of software with respect to known security issues within source code by

1) defining a unified measurable set of weaknesses
2) enabling effective discussions and descriptions
3) promoting the selection and use of software security tools and services that can find weaknesses



Clarifying software weaknesses in different areas (1)

1. **Systems Development Manager issue areas**
   - What are the software weaknesses I need to protect against at the level of architecture, design, code?
   - Can I look through the issues by technologies, risks, severity?
   - What have the pieces of my system (COTS packages, internal development, open source) been vetted for?
   - Identify tools to vet code based on tool coverage. How effective are the tools?
2. **Assessment Tool Vendors issue areas**
   - Express what my tool does and identify areas I should expand coverage
3. **COTS Product Vendor issue areas**
   - What have I verified my applications for?
   - What do my customers want me to verify for?



Clarifying software weaknesses in different areas (2)

4. **Researcher issue areas**
   - Quickly understand what is known
   - Easily identify areas to contribute/refine/correct
5. **Educator issue areas**
   - Train students with the concepts they will use in practice
6. **Operations Manager issue areas**
   - What issues have my applications been verified for? (COTS/Internal/OS)
   - What types of issues are more critical for my technology?
   - What types of issues are more likely to be successfully exploited?



Common Vulnerabilities and Exposures (CVE)

CVE® is an international, community-based effort that maintains a community-driven, open data registry of publicly known cybersecurity vulnerabilities (**CVE List**)

The **Common Vulnerabilities and Exposures** identifies, defines, and classifies publicly disclosed cyber vulnerabilities. CVE provides a reference-method for publicly known cyber security vulnerabilities and exposures

- The National Cybersecurity Federally Funded Research and Development Center (FFRDC) maintains the system
- The system is operated by **The MITRE Corporation** with funding from the National Cyber Security Division of the United States Department of Homeland Security
- The Mitre Corporation functions is the editor and the primary **CVE Numbering Authority** (**CNA**)



CVE program

- Transition from a hub-and-spoke to a federated governance and operational model to keep pace with the proliferation of vulnerabilities
- Produce **CVE Records** for products not covered by another *CVE Numbering Authority* (CNA)
- Adjudicate disputes for CVE Records and CNA scope issues
- Establish and implement operational guidance
- Maintain the program infrastructure
- Moderate stakeholder discussions



Motivation

- Difficult to correlate data across multiple organizations and tools, such as
  - Intrusion Detection Systems (IDS)
  - Assessment tools
  - Security tools
- Incident information
- Difficult to conduct a detailed comparison of tools or databases
- Vulnerabilities are counted differently. Which is more comprehensive?



CVE: *One common language*

- Lists all publicly known security problems
- Assigns unique identifier to each problem
- Remains independent of multiple perspectives
- Is publicly open and shareable
- Community-wide effort via the CVE Editorial Board
- In Oct. 2022, there were [redacted] CVE Records accessible
- In Oct. 2026, there are over [redacted] CVE Records



CVE is

- the industry standard for vulnerability and exposure names one standardized description for each vulnerability or exposure
- **a dictionary rather than a database** → How disparate databases and tools can speak the same language
- the way to interoperability and better security coverage
- a basis for evaluation among tools and databases
- free for public download and use
- industry-endorsed via the CVE Numbering Authorities, CVE Board, and CVE-Compatible Products



CVE identifiers

CVE IDs have the format CVE-YYYY-NNNNN

- The **YYYY** portion is the year that the CVE ID was assigned OR the year the vulnerability was made public
- The **Description** portion of CVE Identifier (CVE ID) entries are typically written by CVE Numbering Authorities (CNAs), MITRE's CVE Content Team, or individuals requesting a CVE ID
- To write a CVE Description, the MITRE CVE Content Team analyzes public, third-party reports of vulnerabilities (*references*); extracts the relevant information from each reference; resolves any conflicting information or inconsistent usage of terminology; and then writes the description
- Each CVE Identifier includes appropriate References that
  - identify the source
  - include an identifier to facilitate searching on a source's website
  - note the associated CVE Identifier



State of CVE IDs

- **RESERVED** → it has been reserved for use by a CVE Numbering Authority (CNA)
  - The CVE is populated with details and published on the CVE List
  - It will become available in the U.S. National Vulnerability Database (NVD)
  - As one of the final steps in the process, the NVD Common Vulnerability Scoring System (CVSS) scores for the CVE ID are assigned by the NIST NVD team
- **DISPUTED** → When one party disagrees with another party's assertion that a particular issue in software is a vulnerability
- **REJECT** → Not accepted as CVE



Example: CVE identifiers

A CVE entry contains:

- CVE ID
- Description
- References

CVE-ID

CVE-2016-3191 Learn more at National Vulnerability Database (NVD)
• Severity Rating • Fix Information • Vulnerable Software Versions • SCAP Mappings

Description

The compile_branch function in pcre_compile.c in PCRE 8.x before 8.39 and pcre2_compile.c in PCRE2 before 10.22 mishandles patterns containing an (*ACCEP substring in conjunction with nested parentheses, which allows remote attackers to execute arbitrary code or cause a denial of service (stack-based buffer overflow) via a crafted regular expression, as demonstrated by a JavaScript RegExp object encountered by Konqueror, aka ZDI-CAN-3542.

References

Note: References are provided for the convenience of the reader to help distinguish between vulnerabilities. The list is not intended to be complete.

- CONFIRM:http://vcs.pcre.org/pcre2?view=revision&revision=489
- CONFIRM:http://vcs.pcre.org/pcre?view=revision&revision=1631
- CONFIRM:https://bugs.debian.org/815920
- CONFIRM:https://bugs.debian.org/815921
- CONFIRM:https://bugs.exim.org/show_bug.cgi?id=1791
- CONFIRM:https://bugzilla.redhat.com/show_bug.cgi?id=1311503
- CONFIRM:http://www.oracle.com/technetwork/topics/security/linuxbulletinapr2016-2952096.html



Memory safety vulnerabilities

Memory safety vulnerabilities are the most prevalent type of disclosed software vulnerability. They affect a computer's memory in two primary ways:

1) Programming languages like C allow programmers to manipulate memory directly, making it easy to accidentally introduce errors in their program that would enable a seemingly routine operation to corrupt the state of memory

2) Memory safety issues can arise when a programming language exhibits an "undefined behavior." This happens when the programming language standard provides no specification or guidance on how the program should behave under conditions not explicitly defined in the standard

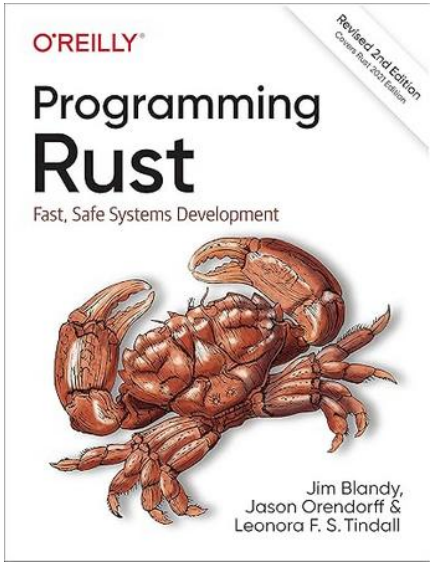


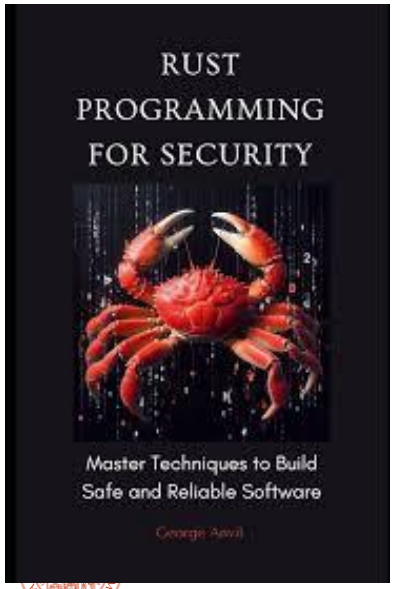
Some statistics

- About 70% of **Microsoft** common vulnerabilities and exposures (CVEs) are memory safety vulnerabilities
- About 70% of vulnerabilities identified in **Google's Chromium** project are memory safety vulnerabilities
- In an analysis of **Mozilla** vulnerabilities, 32 of 34 critical/high bugs were memory safety vulnerabilities
- Based on analysis by **Google's Project Zero** team, 67 percent of zero-day vulnerabilities in 2021 were memory safety vulnerabilities



Rust: *A safer language*





It is not enough to rely on bug-finding tools. The preferred approach is to use “safe” programming languages that can reject unsafe programs at compile time, thereby preventing the emergence of memory safety issues → Rust boasts a unique security model, which **promises memory safety and concurrency safety**, while providing the performance of C/C++ that are memory-unsafe. As a consequence, programming mistakes can result in memory corruption that often results in security vulnerabilities

Rust **catches mistakes at compile time** that would have resulted in memory corruption and other undefined behaviors at runtime, without sacrificing the performance or low-level control of these languages

Information security (*the traditional* IT *goals*!)

Confidentiality

- Assurance that information is shared only among authorized persons or organizations

Integrity

- Assurance that the information is authentic and complete.
- Maintaining and assuring the accuracy and consistency of data over its entire life-cycle

Availability

- Assurance that the systems responsible for delivering, storing and processing information are accessible when needed, by those who need them



Operational security (*the OT goals*)

From January 2027
Regul. EU 2023/1230

Safety

- **It is the first must for any society and industry**

Availability

- Assurance that the systems responsible for delivering, storing and processing information are accessible when needed, by those who need them

Integrity

- Assurance that the information is authentic and complete.
- Maintaining and assuring the accuracy and consistency of data over its entire life-cycle

Confidentiality

- Assurance that information is shared only among authorized persons or organizations

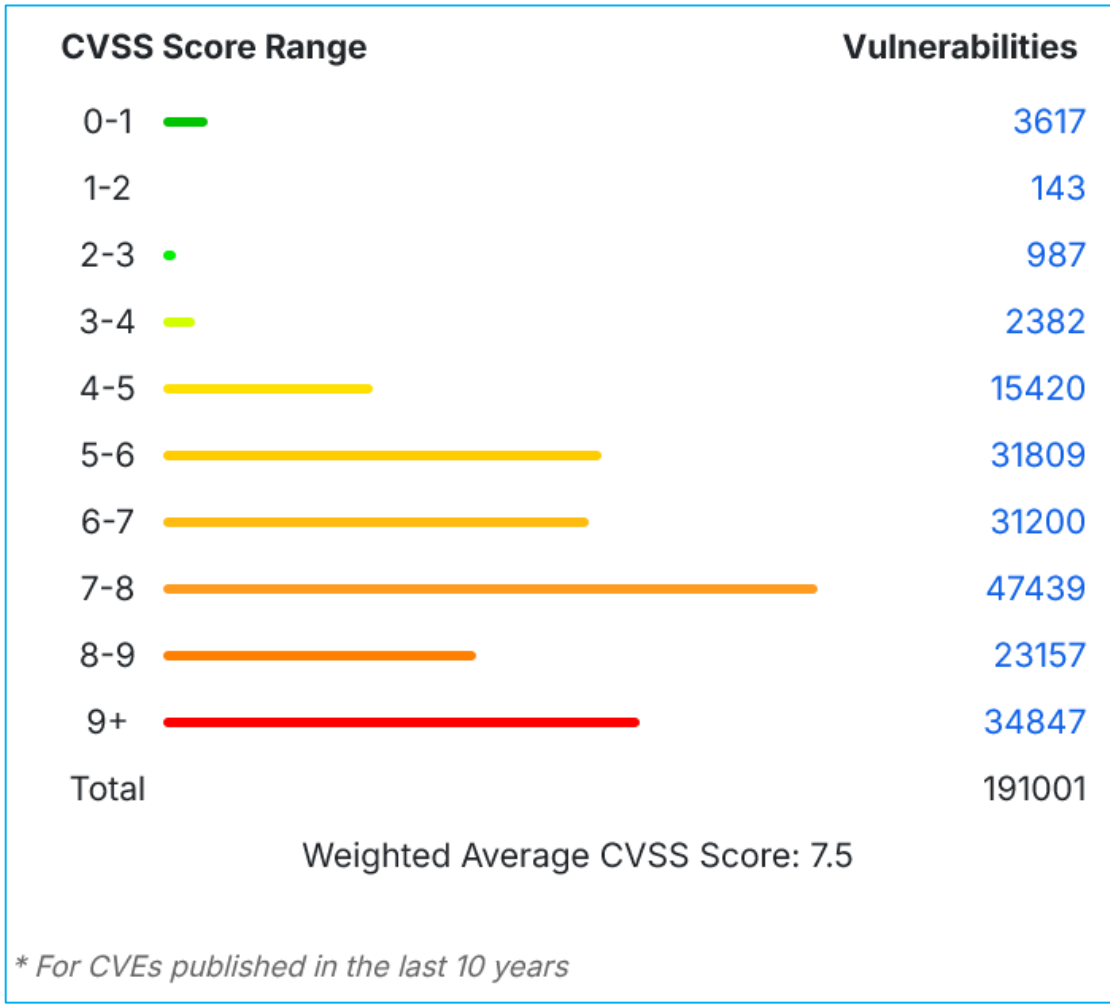


Software vulnerabilities

Vulnerability in software is a major source of insecurity

- 20 flaws per thousand lines of code (Dacey 2003)
- Exponential increase in known vulnerabilities
  - 171 in 1995, 3784 in 2003, >200k today

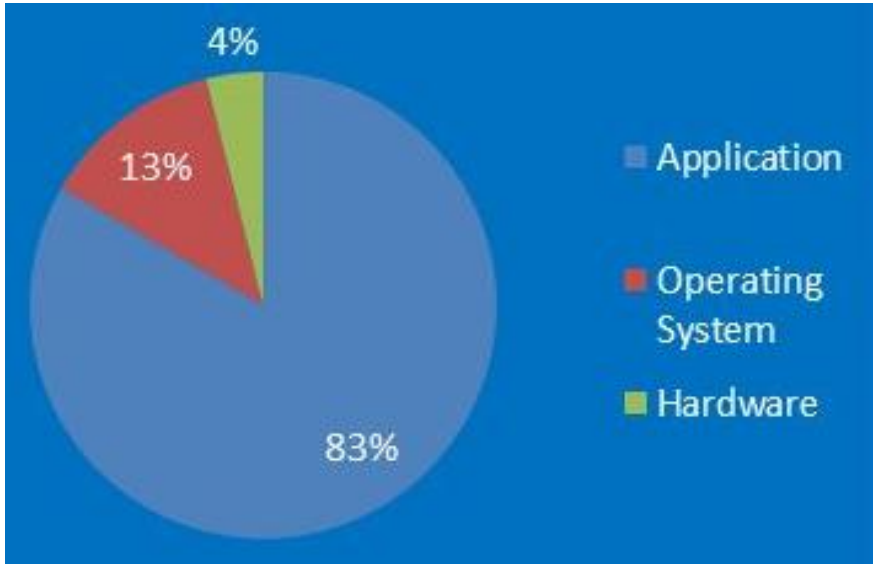
- **Fully secure software is unlikely**
  - Blaming just software vendors is not a solution, vulnerabilities are inevitable
- **95% of breached could be prevented by keeping systems up-to-date with patches** (Dacey 2003)



Malicious Software

There have been at least 360,000 new malicious files detected every day in 2017—an 11.5% increase from the previous year.

According to Kaspersky Lab's Number of the Year for 2017, a number of these new malicious files (processed by the company's in-lab detection technologies) fall into the malware category (78%); however, viruses still account for 14% of daily detections. The remaining files are advertising software (8%).



Patches

- When vendors learn of vulnerabilities, they can release **patches**
- Unfortunately, users and system administrators tend not to install the patches immediately, thus leaving the systems vulnerable to attackers who can access the system with the appropriate attack vector



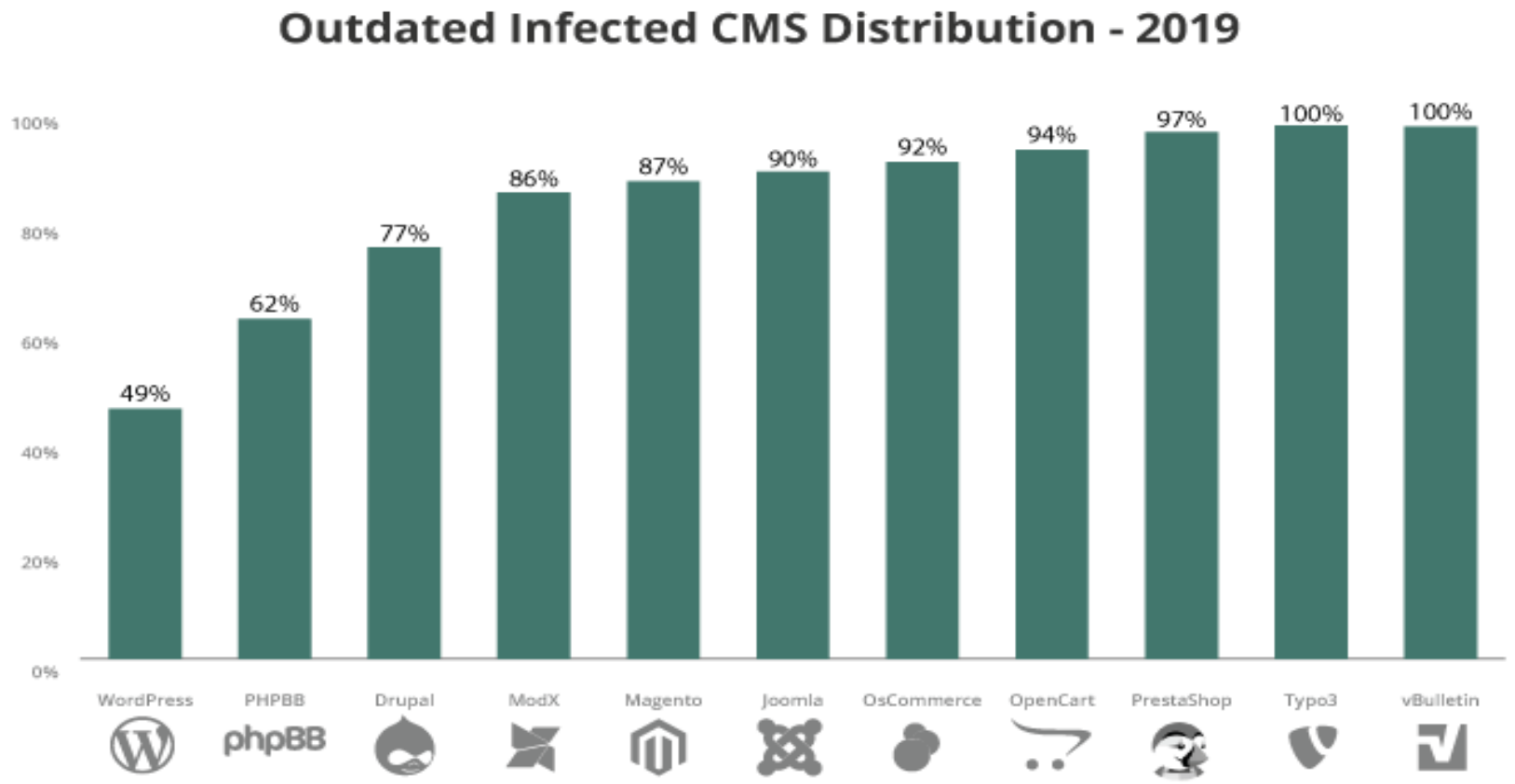
For example,

- the **Conficker worm** has infected close to 7M computers around the world, yet the patch to prevent infection has been widely available (and not used by many) for close to a year
- **Equifax** was penetrated at the end of **July**, but the vulnerability was disclosed, and the related patch was available in **March**

Vulnerable and outdated components

Even simple applications have a lot of dependencies. Failing to update every piece of software on the backend and the (typical) Web frontend of a service will introduce cybersecurity risks.

For example, more than **56%** of all **CMS** applications are out of date



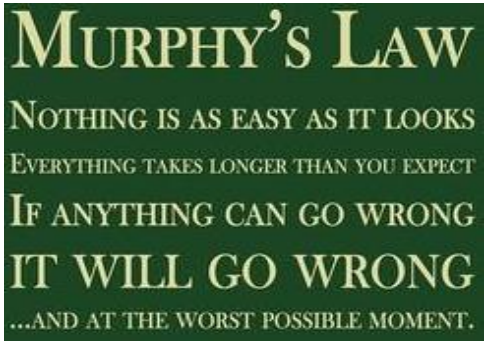
Main questions

Why are not we updating our software on time?

Why is this still such a huge problem today?

Some possibilities

- Webmasters/system administrators cannot keep up with the excessive number of updates. After all, updating properly applications of a company (not a PC) takes a lot of time
- Legacy code could not work on newer versions of its dependencies
- Administrators do not have the expertise to properly apply updates
- Administrators are scared that something breaks on their service(s)
- …



Applications are considered vulnerable if

1) **The software is vulnerable, unsupported, or out of date**. This includes the OS, Web/application server, database management system (DBMS), applications, APIs and all components, runtime environments, and libraries
2) You do not know the versions of all components you use (both client-side and server-side). This includes components you directly use as well as *nested dependencies*
3) You do not fix or upgrade the platform, frameworks, and dependencies in a timely fashion. This happens in environments when patching is a monthly or even quarterly task under change control, which leaves organizations open to many days or months of unnecessary exposure to software vulnerabilities
4) The software developers do not test the compatibility of updated, upgraded, or patched libraries
5) You do not secure the components' configurations



The disclosure issue

Full Vendor Disclosure

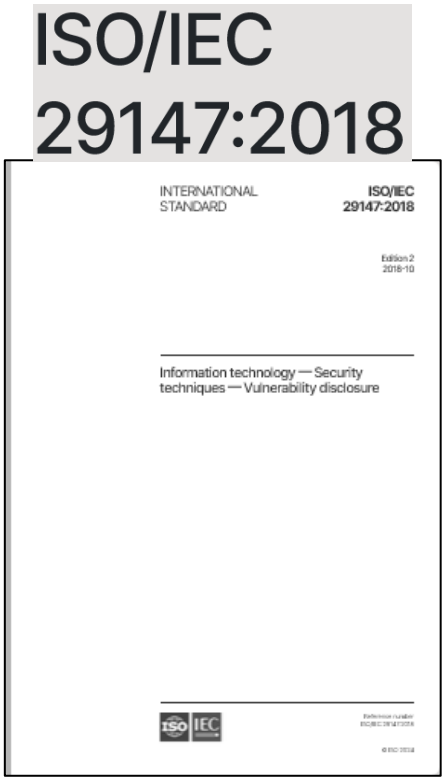
- Promotes *secrecy*
- Gives full control of the process to the vendor

Immediate Public Disclosure

- Promotes *transparency*
- Gives the vendor a strong incentive to fix the problem
- Allows vulnerable firms to take intermediate measures

Hybrid Disclosure

- Promotes both *secrecy* and *transparency*



Known vulnerability is a terrible risk

Every time you disregard an update warning, you are allowing a known vulnerability to survive in your system.

Competent threat actors analyze software vulnerabilities and are quick to investigate software vulnerabilities and changelogs made (or not made) to a project. And if you did not patch it, then …



Zero-day vulnerability is a different problem

Zero-day vulnerability

- *Def. 1* - A zero-day vulnerability is a vulnerability in a system or device that has been disclosed but is not yet patched



- *Def. 2* - **A zero-day vulnerability is a vulnerability in a system or device that has been found, not yet disclosed to the vendor and, therefore, not yet patched**



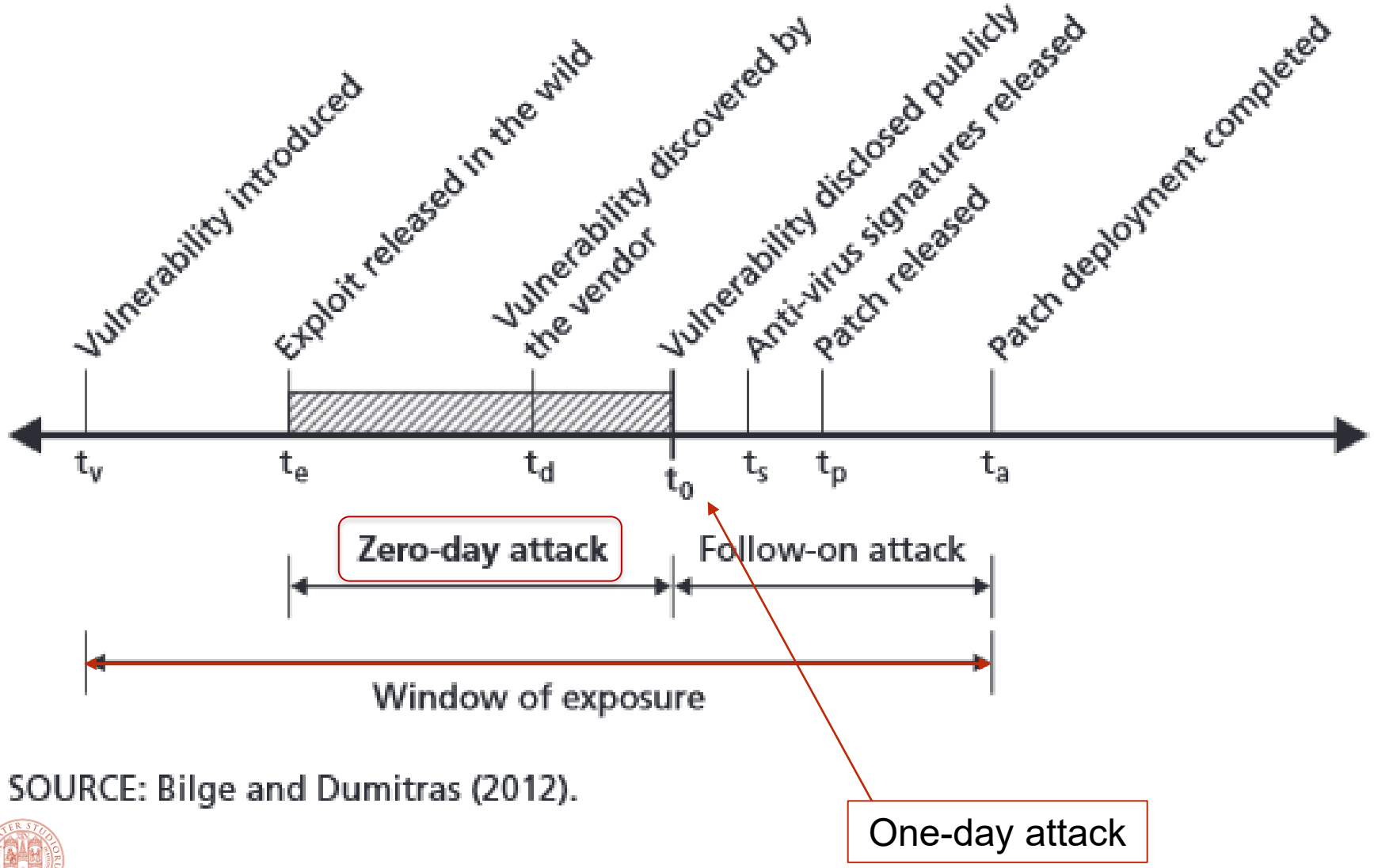
- The most competent attackers (e.g., APT) exploit zero-day vulnerabilities, when necessary
- However, APT and other threat actors can use also **known vulnerabilities (>0 day)** for most attack campaigns because many organizations are “lazy to patch” or “unable to patch”

Software vulnerability: a long-life cycle

1. **Discovery of vulnerability**
   - Malicious software users: Different types of competent attackers
   - Business (e.g., specialized companies, Zerodium)
   - Benign users: Software users, Security firms, Security researchers
2. **Disclosure of vulnerability**
   - How should a benign user disseminate vulnerability information?
     - Should it be kept secret?
     - Should the public be informed immediately?
3. **Patching of vulnerability**



Life cycle of a software vulnerability and 0-day attacks



SOURCE: Bilge and Dumitras (2012).

4. - People tend to forget



Do you remember any of the following disasters?

- **WannaCry?**
- **Not Petya?**
- **Equifax?**
- **REvil (Ransomware Evil)?**
- **Crowdstrike?**

4) FORGETFULNESS OF USERS

Example: the (in)famous Equifax data breach

In July 2017 Equifax disclosed that “hackers infiltrated its network hackers and stole the personal information of **147.7 million Americans** from its servers”

This represents more than half the US adult population:

- Customer names
- Social Security numbers
- Birthdates
- Addresses
- Ratings
- …



Equifax is a powerful *Consumer Reporting Agency*

- It assembles and evaluates consumer credit information or other consumer information for the purpose of furnishing consumer reports to other businesses and organizations that use the information to assess or evaluate creditworthiness
- A lender can use the information provided to determine:
  - whether or not to offer credit to an individual
  - the rate of interest to be assigned to the loan
  - and other terms of the contract
- Other entities use this information to help make decisions about individuals' credit worthiness when determining eligibility for insurance, housing, employment
- The customers include banks, insurances, federal agencies, such as the Internal Revenue Service, Social Security Administration, U.S. Postal Service, Federal Trade Commission, the Bureau of Consumer Financial Protection

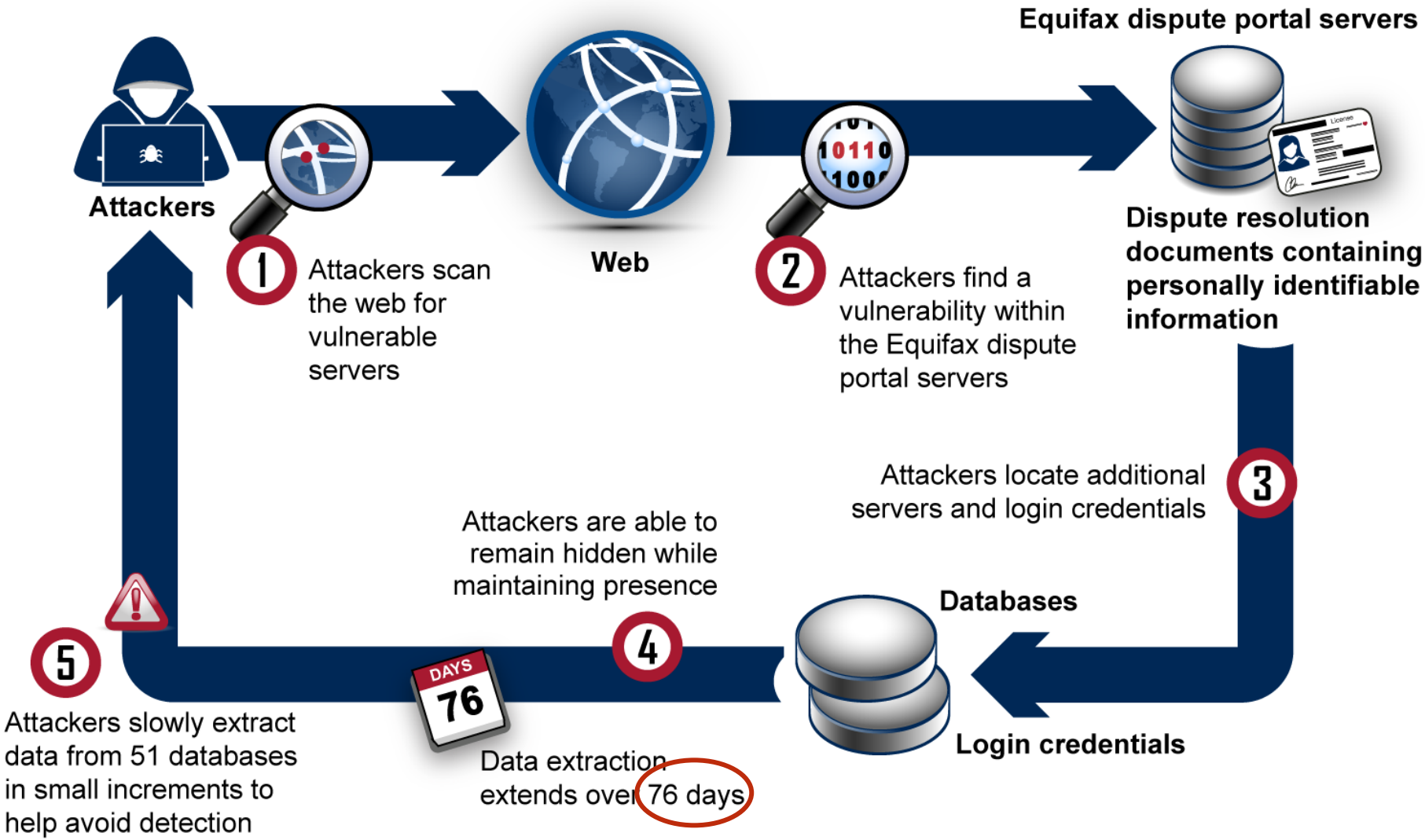


Timeline of the events

- On **March 7, 2017**, a vulnerability of ***Apache Struts Web Framework*** was published with related patch. All users were urged to update it
- Between **April and May 2017** unidentified individuals discovered the presence of this known vulnerability in software running on Equifax's online *dispute portal* that could be used to obtain access to the system
- On **May 12, 2017**, attackers (suspect: *Chinese military officers*) exploited the vulnerability and began to extract data containing accounts of other Equifax's information systems, and they moved there
- According to Equifax, the attackers used several techniques to disguise their exploit of the Equifax systems and the database queries they conducted. Activities went on for **76 days** with no detection
- On **July 29, 2017**, Equifax discovered the breach and reported that it took actions to address the factors that allowed the attackers to successfully gain access to its network
- On **September 7, 2017**, Equifax announced the huge data breach
- In **August 2017**, top managers sold Equifax stocks (*and some were fired*)



Cyber attack to Equifax



Source: GAO, based on information provided by Equifax. | GAO-18-559

Equifax's errors

The vulnerability involved the ***Apache Struts Web Framework*** and would allow an attacker to execute remote commands on affected systems

While **the failure to update Struts was the key failure**, analysis of the breach found further faults in Equifax' system that made it possible for the breach to occur, including:

1) insecure *network design* (insufficient network segmentation and segregation) to avoid/limit lateral movements
2) inadequate *encryption* of Personally Identifiable Information (PII)
3) ineffective breach *detection* mechanisms



Sum of Equifax's errors and related fine

1) Equifax did not check to make sure employees followed through on the **patching process**
2) Equifax failed to detect that a patch was needed because the company used an automated scan that was not properly configured to check all the places that could be using the vulnerable software
3) Equifax failed to update security certificates that had expired 10 months earlier
4) Equifax stored admin credentials and passwords in unprotected plain-text files
5) Equifax did not detect intrusions on "legacy" systems like Automated Computer Inventory System (ACIS)
6) Equifax failed to segment its network to limit how much sensitive data an attacker could steal



The bad news for defenders: The “goldfish memory” of customers





The bad news for defenders: The “goldfish memory” of customers



Again: visit and enjoy various communications channels

https://cve.org/

https://nvd.nist.gov/

For software and cybersecurity professionals:

Tweets by @CVEnew



Another great source for software community: OWASP

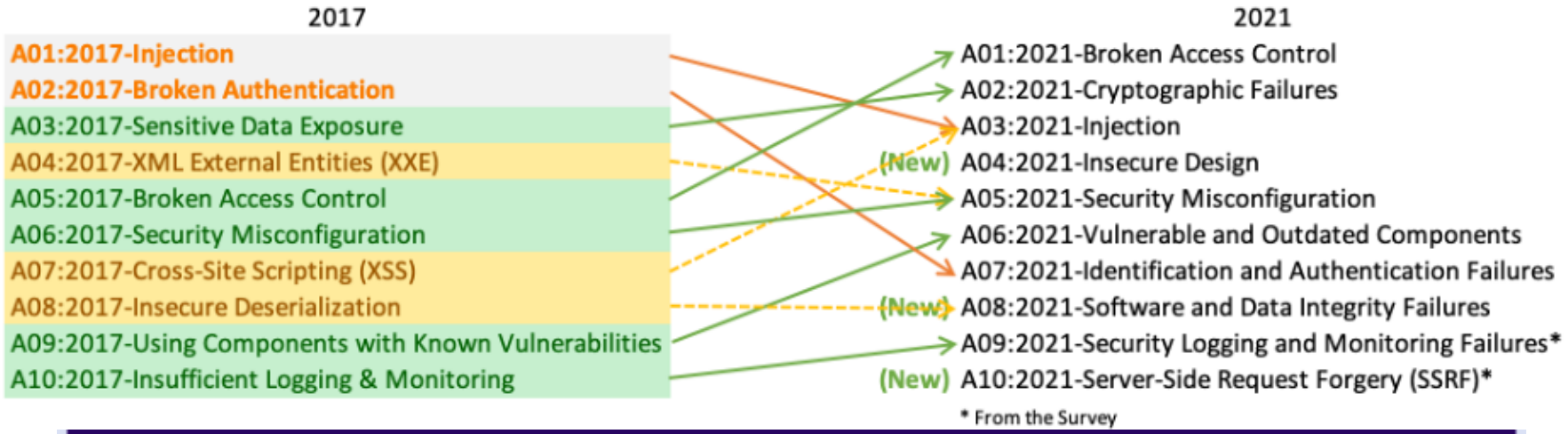
The **Open Web Application Security Project®** (**OWASP**) is a nonprofit foundation, born in 2001, that works to improve the security of software

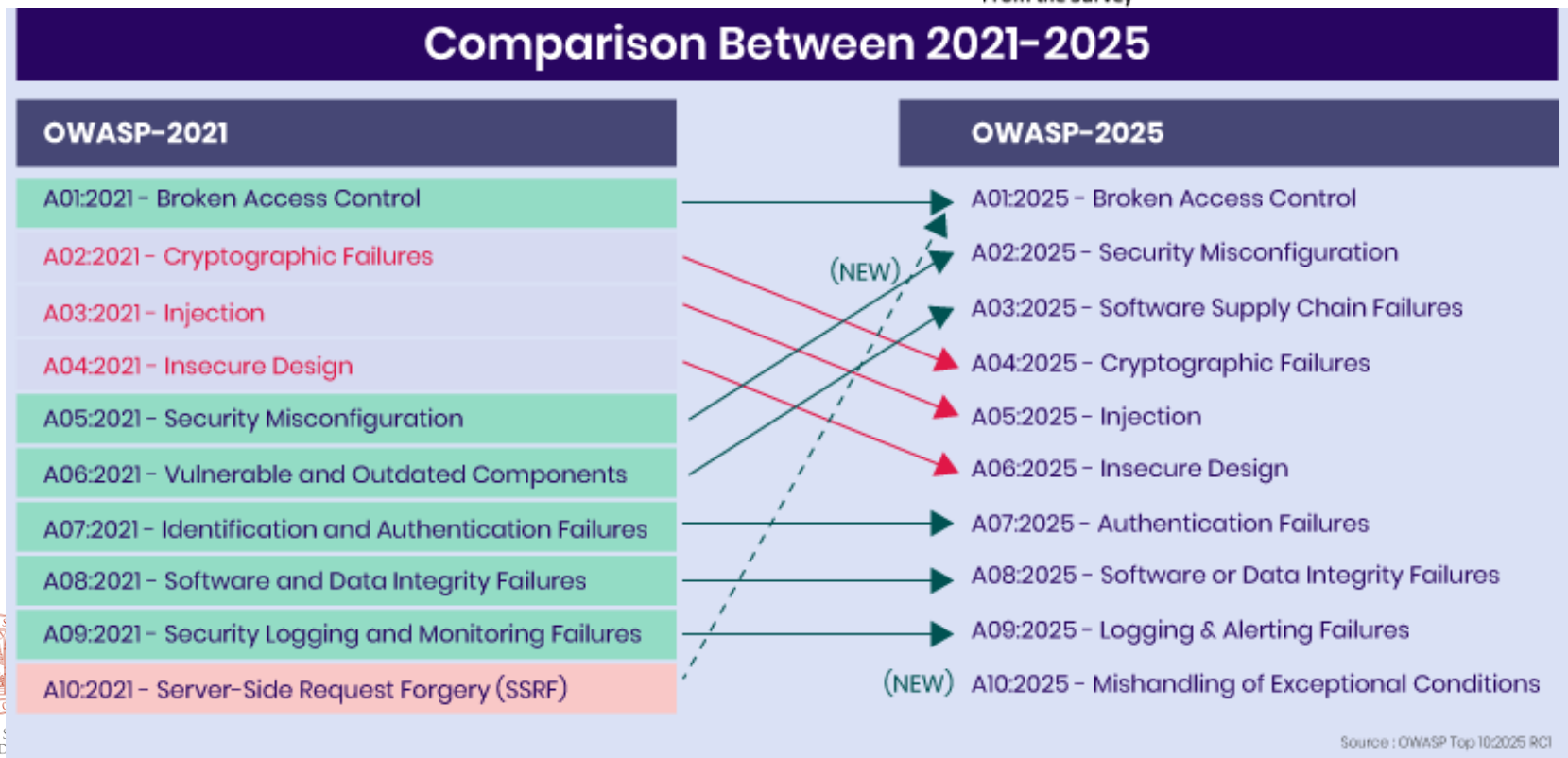
Through **community-led open-source** software projects, hundreds of local chapters worldwide, tens of thousands of members, and leading educational and training conferences, the OWASP Foundation **is the source for developers and technologists to secure the Web**



Join the OWASP Community!
Connect with 50,000+ security professionals worldwide. Whether you are a developer, security researcher, or just passionate about application security, there is a place for you in the OWASP community.

OWASP 2017 → 2019 → 2021 → 2023





The Top 10 OWASP vulnerabilities (2023)



API1:2023 - Broken Object Level Authorization	APIs tend to expose endpoints that handle object identifiers, creating a **wide attack surface** of Object Level Access Control issues. Object level **authorization** checks should be considered in every function that accesses a data source using an ID from the user.
API2:2023 - Broken Authentication	**Authentication** mechanisms are often implemented incorrectly, allowing attackers to compromise authentication tokens or to exploit implementation flaws to assume other user's identities temporarily or permanently. Compromising a system's ability to identify the client/user, compromises API security overall.
API3:2023 - Broken Object Property Level Authorization	This category combines API3:2019 Excessive Data Exposure and API6:2019 - Mass Assignment, focusing on the root cause: the lack of or improper **authorization** validation at the object property level. This leads to information exposure or manipulation by unauthorized parties.



The Top 10 OWASP vulnerabilities (2023)

API4:2023 - Unrestricted Resource Consumption	Satisfying API requests requires resources such as network bandwidth, CPU, memory, and storage. Other resources such as emails/SMS/phone calls or biometrics validation are made available by service providers via API integrations and paid for “per request”. Successful attacks can lead to Denial of Service or an increase of operational costs.
API5:2023 - Broken Function Level Authorization	Complex access control policies with different hierarchies, groups, and roles, and an unclear separation between administrative and regular functions, tend to lead to **authorization flaws**. By exploiting these issues, attackers can gain access to other users’ resources and/or administrative functions.
API6:2023 - Unrestricted Access to Sensitive Business Flows	APIs vulnerable to this risk expose a business flow - such as buying a ticket, or posting a comment - without compensating for how the functionality could harm the business if used excessively in an automated manner. This doesn't necessarily come from implementation bugs.



The Top 10 OWASP vulnerabilities (2023)

API7:2023 - Server Side Request Forgery	Server-Side Request Forgery (SSRF) flaws can occur when an API is fetching a remote resource without validating the user-supplied URI. This enables an attacker to coerce the application to send a crafted request to an unexpected destination, even when protected by a firewall or a VPN.
API8:2023 - Security Misconfiguration	APIs and the systems supporting them typically contain complex configurations, meant to make the APIs more customizable. Software and DevOps engineers can miss these configurations, or do not follow security best practices when it comes to configuration, opening the door for different types of attacks.
API9:2023 - Improper Inventory Management	APIs tend to expose more endpoints than traditional web applications, making proper and updated documentation highly important. A proper inventory of hosts and deployed API versions also are important to mitigate issues such as deprecated API versions and exposed debug endpoints.
API10:2023 - Unsafe Consumption of APIs	Developers tend to trust data received from third-party APIs more than user input, and so tend to adopt weaker security standards. In order to compromise APIs, attackers go after integrated third-party services instead of trying to compromise the target API directly.



Some common vulnerabilities

- **Broken access control**
- **Identification and authentication failures**
- **Code injection**
- **Insecure design**
- **Security misconfigurations**
- **Software and data integrity failures**
- **Security logging and monitoring failures**
- **Server-side request forgery**
- **Cross-site scripting**



Broken access control

- **Access control is how an application grants users' access to some resources, but not to others**. Access control should put a limit on what sections or pages visitors can reach, depending on their needs
- For example, in an e-commerce site, you probably need access to the ***admin* panel** in order to add new products or to set up a promotion for the upcoming holidays. A broken access control occurs when you allow the rest of your visitors to reach your login page thus opening your site to attacks
- **Broken access control is an Authentication and/or Authorization issue** (detected in more than 90% of Web-based applications)



Categories of resources

Resources fall at least into three categories:

1) Functions that can modify data on the server, or even modify the server's functionality which should only be accessed by the system administrator(s)
2) Sensitive data, which should only be accessed by few select users
3) The public data and services

Authorization checks are performed after **authentication** (that is, *log in*) to verify whether a user is authorized or not to gain access to a resource. **It is a management problem**



Broken access control

Broken access control is a common problem with almost all major **Content Management Systems** (CMS) because, by default

- they give worldwide access to the admin login page
- most of them do not force you to establish a two-factor authentication method (2FA)

Broken access control can occur everywhere

- Access to a hosting control / administrative panel
- Access to a server via FTP / SFTP / SSH
- Access to a website's administrative panel
- Access to other applications on your server
- Access to a database
- Attackers can exploit authorization flaws to the following:
  - Access unauthorized functionality and/or data
  - View sensitive files
  - Change access rights

Broken access control vulnerabilities can be expressed in many forms through almost every Web technology



Possible attack scenarios: *ID*

- When looking for something in a database, most of the time we use a unique ID
- Often in the past (less frequent today), this ID is used in the URL to identify what data the user wants to get
- When I go to my own profile page, the URL looks something like this: **`https://example.com/profile?id=34887`**
- This page might contain sensitive data, which nobody else should see
- If I replace the ID with another user's ID and the Web server is configured improperly, then I can visit another page. Example: using **`https://example.com/profile?id=33142`**, I will get the profile page of another user, with all of his/her sensitive data



Can CWE and CVE be used to break into my network, system, or product?



Different opinions

Security through obscurity does not work

[Software & Cybersecurity world]

“Loose lips sink ships”

[Military & Intelligence world]



Software and Cybersecurity

- Security should not depend on secrecy
- Open Source Perspective & Disclosure Helps Defenders
- Attackers learn little or nothing from public disclosure
- Disclosures prompts designers to improve the defense -- learn of flaws and fix
- Disclosure prompts other defenders/users of software to patch and fix

Costs of disclosure are low. Benefits are high



Military and Intelligence

- It is hard for attackers to get close enough to learn the physical defenses
- Disclosure teaches the designers little about how to improve the defenses
- Disclosure prompts little improvement by other defenders

Costs from disclosure are high. Benefits are few



Differences in cybersecurity

- Many attacks
- Attackers learn from previous attacks
- Attackers communicate about vulnerabilities (and can do easily at Internet time)
- Because of attackers' distributed knowledge, disclosure often helps defenders more than attackers for cyber attacks

- Cryptography
  - **Kerckhoff's principle**: "A cryptosystem should be secure even if everything about the system, except the key, is public knowledge"
  - **Diffie's point** about keys and encryption algorithms



In favor of disclosure

- **CWE and CVE allow developers to minimize weaknesses as early in the lifecycle as possible, improving overall security**
- They help reduce risk industry-wide by enabling more effective community discussion about finding and mitigating these weaknesses in existing software and hardware, and reducing them in future updates and releases
- They enable more effective description, selection, and use of **security tools and services** that organizations can use to find these weaknesses and reduce their risk now
- There is a shift in community opinion towards sharing information, as reflected in the success of the collaborative nature of similar programs such as CWE, CVE, CVSS, CAPEC. These communities include key organizations in cyber security



Against disclosure

- **Attackers** have time, clear incentives and no constraints to exploit vulnerabilities

- **Defenders** have temporal and resource limits that prevent overall fixing. They have less motivations and must choose their daily priorities



Further study

"A Theory of Disclosure for Security and Competitive Reasons: Open Source, Proprietary Software, and Government Agencies" [Peter Swire, Georgia Institute of Technology, Jan. 2006]

